code → functionality } libraries

ftp → files

email → messages

telnet → remote comedian
ssh

Program

Libraries are now in API's (Application Programming Interface)

If it's static → Libraries

If I want to use someone else it's in API, they're dynamic, in the cloud

2010 ⊖ Mashup: Combination of services

After 2010: Disapeared and their API's died, that afected other apps that used those API's negatively, some functionalities stoped working Ex. yahoo weather, their API seems to have been created again, and there are alternatives

We can still make mashups, but keep in mind API's could dissapear some day.

Program autorizations first (log in, password)

OAuth

Web service, method of comunication for web app

Run on application layer port 80

OSI model

nowdays: TCP/IP model → { app / transport / network } don't need to know

HTTP protocol most used ~~port 80~~ port 443 (HTTPS) for security

VERZATIL

search already busy ports "netstat -rn"

SMTP . emails

Message formats: 90% +

SRI – xml (JAX-B), JSON (Gson), RSS (news)

MIME types: the types the browser supports. html, image (jpeg, png), mp3, mp4, docx. – browser/download

Web 1.0 (1991 to 2004) php solved almost everything. simple everyone was happy

- Static pages
- Content in server's filesystems rather than in Databases
- Common Gateway Interface (CGI) instead of web app
- HTML 3.2
- Browser war → Netscape, I. explorer, Firefox, Chrome, Lynx (command line)
- HTML forms sent via email
- Sites aren't interactive

MySQL → Postgres, mySQL / mariaDB

no MIME types

Tim Berners-Lee created www before: images, mail, ftp

197 – 981 js, java script

CGI: tedious for big projects → php starts

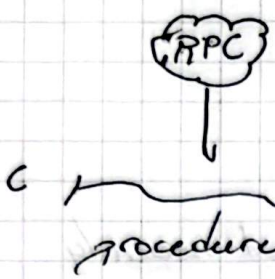
The bases of web 2.0 (Participatory and Social Web)

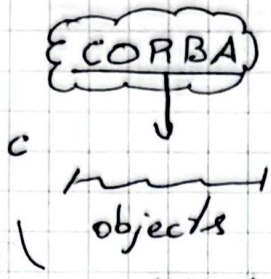
- Public APIs are published by web 2.0 providers in SOAP, REST, RPC. js architectural styles, using different data formats

Private APIs are used internally.

CORBA: Common object request broker architecture

RPC: Remote proced call





IDL: Interface definition language

5-10 years market

REST appeared

same language both ways

- Request/Response formats:
  - SOAP, XML
  - JSon
  - others
- Architectural styles

MASHUPS

- Public APIs + local APIs + My Application
- Don't develop what is done, use others resources
- Low coupling. But APIs may disappear at any moment.

Everybody creates APIs:

Google Mapps, Facebook, Twitter, etc.

Tecnology

- Tools, Languages, Platforms: Java, .Net, Python, etcetera
- Distributed Objects: JEE-EJBs, .Net Remoting: ORMs: Persistent Units, JPAs: Linq, Hibernate, Eclipse Link
- Clients: desktop, gadgets, widgets mobiles, web, Ajax, JavaFX, Angular js... Client-cache.
- Web Frameworks (web 2.0 and beyond, MVC):

  Server-side: Cake PHP, Django, Ruby on Rails, Flask, ASP.NET.Core, Laravel

- Client-side: Angular
- Previously full stacks:
  ASP. Net.
- Architectures:
  - MVC
    - Push-based
  - Three-tier organization
- You use mostly a single programming language
  - Laravel: php
  - Spring MVC: Java
  - Angular 1.8: Java Script
  - Angular Type Script

Distributed security

web 3.0

blockchain, ai, decentralization

Google has user, password → token - open session

Consult. 2 paragraphs → code

Login, menu, redirec. 3 ~~cap~~ screenshots